Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives et algorithmique

Synthèse exécutive : Ce cours aborde les structures répétitives (boucles) en algorithmique et en langage C, indispensables pour automatiser l'exécution d'un bloc d'instructions sans duplication de code, conformément au principe DRY (Don't Repeat Yourself). Il détaille la syntaxe, le fonctionnement et le choix contextuel entre les trois boucles fondamentales : 'while' (tant que), 'do-while' (faire... tant que) et 'for' (pour). Une attention particulière est portée à la prévention des boucles infinies, à la gestion rigoureuse des conditions d'arrêt et aux pièges classiques d'évaluation en BUT Informatique.

1. Utilité des structures itératives et principe DRY

En programmation, répéter des instructions ne doit jamais se faire par duplication manuelle de code. Si un programmeur doit traiter des milliers de données, copier 10 000 fois la même ligne rend le code illisible, impossible à maintenir et source de bogues massifs. Les boucles permettent d'exécuter un bloc d'instructions de manière répétée tant qu'une condition logique est vraie. Le risque majeur est la boucle infinie, qui survient si la condition d'arrêt ne devient jamais fausse en raison d'une absence de mise à jour de la variable de contrôle.

- Principe DRY (Don't Repeat Yourself) : interdiction absolue de dupliquer du code par copier-coller
- Condition d'arrêt : expression booléenne déterminant la poursuite ou la sortie de la boucle
- Variable de contrôle / Compteur : variable mise à jour à chaque itération pour faire progresser la condition

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'emballe. Si le capteur ne détecte plus rien ou si le quota est atteint, le robot s'arrête. Sans condition de fin, le robot continuerait à tourner à vide et détruirait la machine.*

2. La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est utilisée lorsque le nombre d'itérations n'est pas connu à l'avance, car il dépend d'un événement dynamique externe (saisie utilisateur, lecture de fichier, signal réseau). Le test logique étant effectué en amont, si la condition est fausse dès la première évaluation, le corps de la boucle est totalement ignoré. Il est impératif de modifier l'état des variables de la condition à l'intérieur du bloc pour éviter une boucle infinie.

- Pré-conditionnement : la condition est évaluée AVANT chaque passage dans le bloc d'instructions
- Intervalle d'itérations : de 0 à N fois (le bloc peut ne jamais s'exécuter si la condition est fausse dès le départ)
- Initialisation préalable : la variable de condition doit obligatoirement être initialisée avant l'entrée dans le while

Cas concret enseignant : *Exemple du contrôle de code PIN au distributeur de billets : la machine demande le code TANT QUE le code est incorrect ET que le nombre d'essais est inférieur à 3. Si l'usager tape le bon code du premier coup, la boucle s'arrête immédiatement.*

3. La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À l'inverse du 'while', la boucle 'do-while' garantit que les instructions internes s'exécutent une première fois avant qu'aucun test ne soit réalisé. Elle est particulièrement adaptée aux interfaces interactives et au contrôle de saisie (validation de bornes numériques). En langage C, l'oubli du point-virgule final constitue l'erreur de compilation la plus fréquente.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions
- Garantie d'exécution : la boucle s'exécute obligatoirement au minimum 1 fois (1 à N itérations)

• Syntaxe C stricte : obligation impérative d'ajouter un point-virgule ';' après la parenthèse fermante de la condition while

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'On ne peut pas vérifier si la saisie d'un étudiant est valide avant qu'il n'ait tapé une valeur au clavier. Le do-while s'impose donc naturellement.'*

4. La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise l'itération en regroupant les trois éléments vitaux (initialisation, condition, mise à jour) au même endroit. Bien que techniquement optionnelles en langage C, ces trois composantes doivent toujours être renseignées pour garantir la lisibilité et la robustesse du code.

• En-tête condensé : réunit initialisation, condition de maintien et incrémentation en une seule ligne `for(init; cond; inc)`

• Usage privilégié : réservée aux situations où le nombre exact d'itérations est connu avant d'entrer dans la boucle

• Portée (scope) : possibilité de déclarer la variable d'itération directement dans l'en-tête de la boucle

Cas concret enseignant : *Métaphore de la course sur piste d'athlétisme : 'Vous savez avant de partir que vous devez faire 5 tours. Vous initialisez le compteur à 1, vous vérifiez si vous avez fait moins de 5 tours, et à chaque ligne d'arrivée franchie, vous ajoutez 1 au compteur.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Parce que la condition d'arrêt (par exemple i <= 10) reste éternellement vraie si 'i' ne change pas. Le processeur répète indéfiniment les mêmes instructions sans pouvoir sortir, provoquant un gel du programme (boucle infinie).

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, absolument. Toute boucle 'for' peut être réécrite sous forme de boucle 'while' en plaçant l'initialisation avant, la condition dans le while, et l'incrémentation à la fin du bloc d'instructions. Le 'for' n'est qu'un sucre syntaxique pour plus de concision.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit valider que N est strictement supérieur à 0 via une boucle 'do-while', puis calculer le cumul au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Réaliser la saisie sécurisée avec do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); pour bloquer toute valeur négative ou nulle. Étape 3 : Initialiser somme = 0. Étape 4 : Mettre en place la boucle de calcul for (i = 1; i <= n; i++) { somme += i; }. Étape 5 : Afficher le résultat final avec printf.

Résultat : Pour N = 5, la boucle effectue 5 itérations (somme prend successivement les valeurs 1, 3, 6, 10, 15). Le programme affiche la valeur finale 15 de manière sécurisée.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : Le 'while' évalue sa condition avant l'exécution (0 à N itérations), tandis que le 'do-while' l'évalue après l'exécution, garantissant au moins 1 passage (1 à N itérations).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès le premier test ?

R : Le bloc d'instructions de la boucle n'est pas exécuté du tout, et le programme passe directement à la suite du code située après la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en C ?

R : L'oubli du point-virgule final juste après la parenthèse fermante de la condition while (ex: `} while (condition);`).